# 1.树的概念

树是由n(n>=0)个结点组成的有限的集合

```
如果n=0 它是一颗空树，这个树的特例
如果n>0, 它是一颗非空树
    特点：
        1.有且仅有一个结点作为树的根结点，简称为根(root)
        2.其余结点可以分为m(m>=0)个互不相交的有限集(T1,T2..Tm)
            其中每一个子集本身又是一颗符合定义的树，称为根结点的子树(subtree)
```

树的结点的组成

```
包含了一个数据元素以及若干个指向子树的分支(保存关系)
```

结点的度(degree)

```
结点拥有的子树的个数，称为结点的度
度为0的结点，称为 叶子结点或终端结点
度不为0的结点，称为 分支结点或非终端结点
```

结点的层次

```
从树开始定义，根为第一层，根的子树为第二层.....
树中结点的最大层次，称为树的高度或树的深度
```

# 2.二叉树binary tree

1.二叉树

```
一种树的结构
特点：
    1.每个结点最多只有两颗子树(不存在度大于2的结点)
    2.二叉树的子树有左右之分，其次序不可以颠倒
```

2.二叉树的5种形态

```
1.空树
2.只有一个结点(没有子结点)
3.只有左孩子
4.只有右孩子
5.有左右两个孩子

问题：3个结点组成的树有多少种形态?
```

3.二叉树的性质

```
1.在二叉树中，第i层最多只有2^(i-1)个结点(i>=1)
    i=1     1    2^0
    i=2     2    2^1
    i=3     4    2^2
    i=4     8    2^3
    ..
    i            2^(i-1)  ：2的(i-1)次方
```

```
2.深度为k的二叉树，最多有2^k-1个结点(k>=1)
    1+2+4+8...+2^(k-1)
    等比数列求和公式
        sn = (a1-an*q)/(1-q)
           = (1-2^(k-1)*2)/(1-2)
           = (1-2^k)/(-1)
           = 2^k - 1    ：2的k次方 - 1
```

```
3.对于任意一颗二叉树，如果其叶子结点的个数n0,度为2的结点的个数为n2,则，n0 = n2+1
    假设一颗二叉树总的结点的个数为N
    度为0的个数 n0
    度为1的个数 n1
    度为2的个数 n2

    N = n0 + n1 + n2       (1)

    设这可二叉树的分支数B  把这棵树从下往上看，每个结点都有一个向上的分支，除了根结点
    B = N-1                (2)

    从上往下看
    度为0结点分支数0     n0*0
    度为1结点分支数1     n1*1
    度为2结点分支数2     n2*2
    B = n0*0 + n1*1 + n2*2 = n1+n2*2  (3)

    =>n0 + n1 + n2 -1 = n1+n2*2
    =>n0 = n2+1
==========================================================
满二叉树：在不改动这棵树的高度的情况下，不能再往这棵树添加结点
    一颗深度为k且有2^k-1个结点的二叉树叫做满二叉树
    特点：最后一层都是叶子结点，其余的每个结点都是度为2的结点
==========================================================
完全二叉树：
    1.除了最后一层外是一颗满二叉树
    2.最后一层的结点插入的顺序先插入上一层结点的左孩子，再插右孩子
    在插入最后一个结点之前，这个二叉树的最后一个结点前面的每个位置都要放结点

满二叉树又是一颗完全二叉树
```

```
4.如果对n个结点的***完全二叉树***，从上到下，从左到右，给每个结点编号，从1开始编号，如果有一个
结点的编号为i,则
    则父节点编号 i/2
    左子节点    i*2
    右子节点    i*2+1
```

```
5.具有n个结点的***完全二叉树***的深度为log2n向下取整+1
    假设深度为k
        前k-1层结点的总数 < n  <=  k层满二叉树结点的总数
            2^(k-1) -1 < n <= 2^k-1
        加1
            2^(k-1) < n+1 <= 2^k
            2^(k-1) <= n < 2^k
        对等式取以2为底的对数
            k-1 <= log2n < k
        得到log2n为k-1,向下取整
        log2n = k-1
        k = log2n+1
```

# 3.二叉树的存储结构

1.顺序存储

```
用一组连续的空间，来保存二叉树的每一个结点
利用性质4 先进行编号，把结点的数据存放到编号对应的数组下标

#define MAXSIZE 1024
typedef int telemtype;
telemtype seqbitree[MAXSIZE];//用来存放二叉树的数据的

seqbitree[1] 根节点
....
seqbitree[i] 编号为i的结点

存储一颗普通的二叉树，补成一颗完全二叉树，再进行编号

优点：访问比较简单，直接访问下标
缺点:数组的大小固定的，可能造成空间的浪费，可能导致空间不足
```

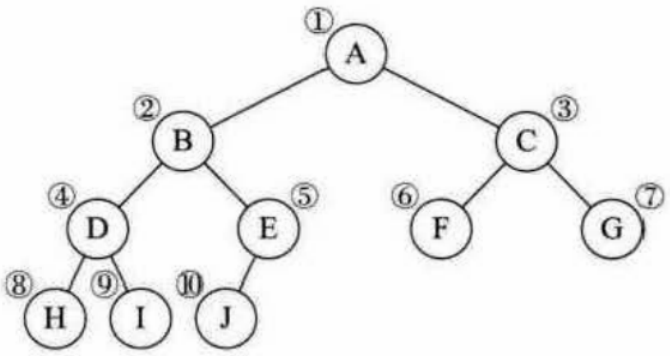



图 6-7-1

将这棵二叉树存入到数组中，相应的下标对应其同样的位置，如图 6-7-2 所示。

| 下标： | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| | A | B | C | D | E | F | G | H | I | J |

2.链式存储

```
typedef int telemtype;
typedef struct binode
{
    telemtype data;//数据域
    struct binode *lchild;//左孩子
    struct binode *rchild;//右孩子
}binode;
```

# 4.二叉排序树

```
二叉排序树(binary sort tree),又称为二叉查找树，它可以是一颗空树，或者是具有下列性质的二叉树：
1.若它的左子树不为空，则左子树上所有的结点的值均小于它的根节点的值
1.若它的右子树不为空，则右子树上所有的结点的值均大于它的根节点的值
3.它的左、右子树也分别是一颗二叉排序树

左 < 根 < 右

 9 56  45  4  5
```

## 4.1创建一颗二叉排序树

```
//插入结点
binode *insert_node(binode *r,telemtype d)
{
    if(r == NULL)//空树
    {
        //新的结点成为你的根结点
        binode *pnew = malloc(sizeof(binode));
        pnew->data = d;
        pnew->lchild = pnew->rchild = NULL;

        r = pnew;
        return r;
    }

    binode *p = r;//p找
    while(1)
    {
        if(p->data > d)//往左找
        {
            //如果p的左子为空，  新的结点就会成为p的左子   直接返回
            if(p->lchild == NULL)
            {
                //创建新结点
                binode *pnew = malloc(sizeof(binode));
                pnew->data = d;
                pnew->lchild = pnew->rchild = NULL;
                //插入
                p->lchild = pnew;//已经插入了，循环结束
                break;
            }
            //否则p要往左走
            p = p->lchild;
        }
```

```
        else if(p->data < d)//往右找
        {
            //如果p的右子为空，  新的结点就会成为p的右子   直接返回
            if(p->rchild == NULL)
            {
                //创建新结点
                binode *pnew = malloc(sizeof(binode));
                pnew->data = d;
                pnew->lchild = pnew->rchild = NULL;
                //插入
                p->rchild = pnew;//已经插入了，循环结束
                break;
            }
            //否则p要往右走
            p = p->rchild;
        }
        else    //相等
        {
            break;
        }
    }

}
//插入结点   递归
binode *insert_node_v2(binode *r,telemtype d)
{
    if(r == NULL)//空树
    {
        //新的结点成为你的根结点
        binode *pnew = malloc(sizeof(binode));
        pnew->data = d;
        pnew->lchild = pnew->rchild = NULL;

        r = pnew;
        return r;
    }

    if(r->data > d)//往左找
    {
        r->lchild = insert_node_v2(r->lchild,d);
    }
    else if(r->data < d)//往右找
    {
        r->rchild = insert_node_v2(r->rchild,d);
    }

    return r;

}
//创建一棵二叉排序树
binode *create_bst()
{
    binode *r = NULL;//保存根结点

    telemtype d;
    while(1)
    {
        //输入数据
```

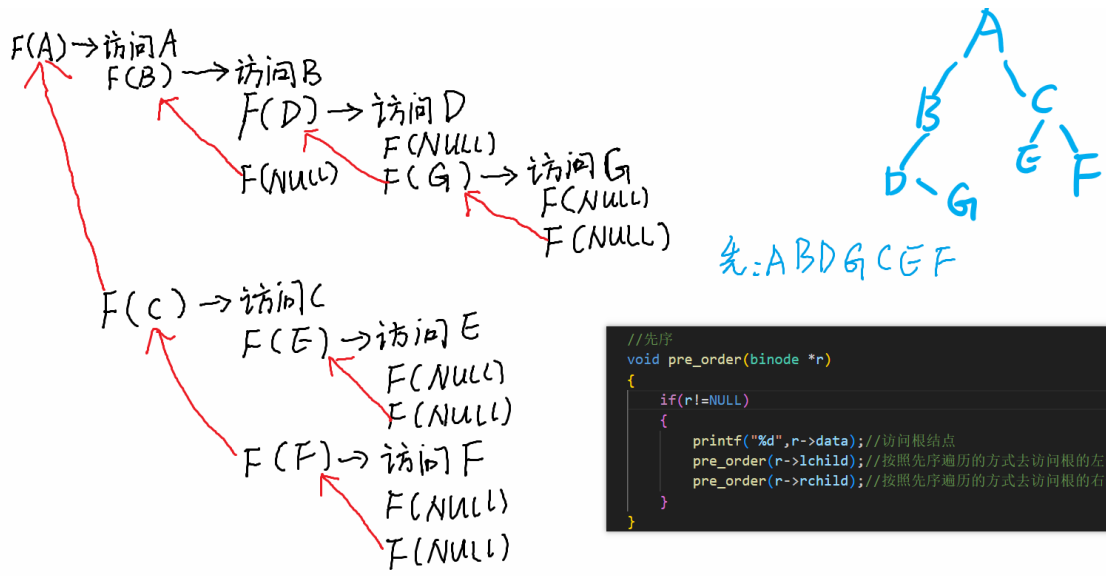
```
        scanf("%d",&d);
        if(d == 0)
            break;

        //插入
        r = insert_node(r,d);
    }

    //返回根结点
    return r;
}
```

## 4.2 二叉树的遍历

二叉树的遍历(从根结点出发，按照某种次序一次访问二叉树的所有结点，使得每个结点仅且被访问一次)

1.先序遍历：根左右

```
1.访问根结点
2.按照先序遍历的方式去访问根的左子树
3.按照先序遍历的方式去访问根的右子树

pre_order(r) -> 用先序遍历的方法访问r   r是树的根结点
1.判断树是否为空
2.访问根结点 printf(r->data)
3.pre_order(r->lchild)
4.pre_order(r->rchild)

//先序
void pre_order(binode *r)
{
    if(r!=NULL)
    {
        printf("%d ",r->data);//访问根结点
        pre_order(r->lchild);//按照先序遍历的方式去访问根的左子树
        pre_order(r->rchild);//按照先序遍历的方式去访问根的右子树
    }
}
```




```
//先序
void pre_order(binode *r)
{
    if(r!=NULL)
    {
        printf("%d",r->data);//访问根结点
        pre_order(r->lchild);//按照先序遍历的方式去访问根的左子树
        pre_order(r->rchild);//按照先序遍历的方式去访问根的右子树
    }
}
```

2.中序遍历:　左根右

```
//中序
void mid_order(binode *r)
{
    if(r!=NULL)
    {
        mid_order(r->lchild);//按照中序遍历的方式去访问根的左子树
        printf("%d ",r->data);//访问根结点
        mid_order(r->rchild);//按照中序遍历的方式去访问根的右子树
    }
}
```

3.后序遍历:　左右根

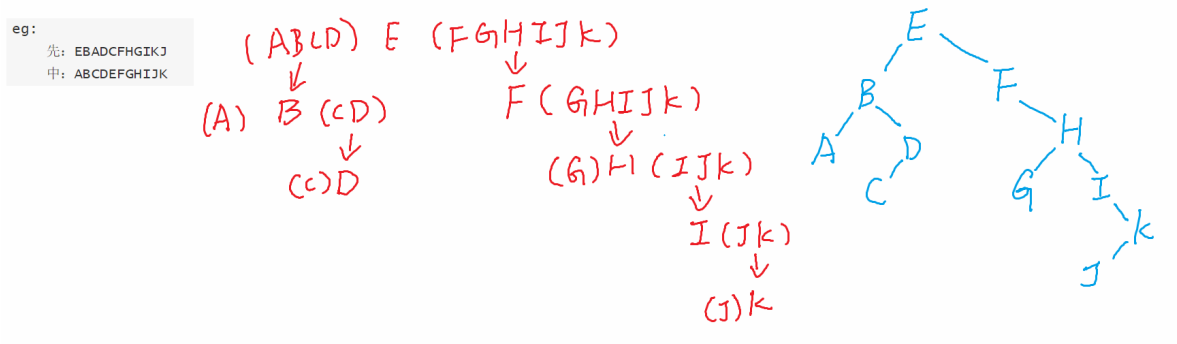
```
//后序
void post_order(binode *r)
{
    if(r!=NULL)
    {
        post_order(r->lchild);//按照后序遍历的方式去访问根的左子树
        post_order(r->rchild);//按照后序遍历的方式去访问根的右子树
        printf("%d ",r->data);//访问根结点
    }
}
```

4.层次遍历(队列做辅助)

```
void level_order(binode*r)
{
    //1.初始化一个队列

    //2.根节点入队

    while()//队列不为空
    {
        //3.出队 出队元素进行访问

        //4.把这个出队结点的左子和右子全部入队
    }

    //5.销毁队列
}
```

练习：给你一颗二叉树的遍历序列，画出这棵树

```
eg:
    先：EBADCFHGIKJ
    中：ABCDEFGHIJK
```




## 4.3求一棵二叉树的深度(递归)

```
没有结点
    高度为0
一个结点：
    获取左子树的高度 0
    获取右子树的高度 0
    返回 左子树的高度和右子树的高度的最大值+1
多个结点：
    获取左子树的高度
    获取右子树的高度
    返回 左子树的高度和右子树的高度的最大值+1
```

## 4.4求一颗二叉树结点的个数(递归)

```
没有结点
    返回 个数为0
一个结点：
    获取左子树结点个数 0
    获取右子树结点个数 0
    返回 左子树结点个数+右子树结点个数+1(自己本身是个结点所以+1)
多个结点：
    获取左子树结点个数
    获取左子树结点个数
    返回 左子树结点个数+右子树结点个数+1(自己本身是个结点所以+1)
```